

Memoria técnica

El Puente Market

Proyecto final - Bootcamp FullStack The Bridge BBK

Versión limpia sin capturas de pantalla

Node.js · Express · Sequelize · PostgreSQL · EJS · Sesiones · JWT

Dato	Contenido
Repositorio	https://github.com/David-LS-Bilbao/el-puente-market
Rama final de referencia	main
Rama de documentación	PUENTE-25/documentacion
Tipo de documento	Memoria técnica académica
Equipo / autores	Jon, Ane, Edy y David
Bootcamp	Bootcamp FullStack The Bridge BBK
Gestión de tareas	Jira - proyecto PUENTE
Fecha	Abril de 2026

Este documento resume el estado final presentado del proyecto, tomando como base el README de la rama final, la rama de documentación y las evidencias técnicas disponibles. Cuando algún dato no aparece documentado con suficiente precisión, se marca como pendiente o mejora futura.

Índice

3. Introducción	4
4. Objetivos del proyecto	4
4.1 Objetivos funcionales	4
4.2 Objetivos técnicos	4
5. Tecnologías utilizadas	4
6. Organización del equipo y proceso de trabajo con GitHub	5
6.1 Uso de Jira para la gestión de tareas	5
6.2 Fases de trabajo	6
6.3 Hitos de integración	6
6.4 Criterio de integración	6
7. Arquitectura de la aplicación	6
7.1 Estructura relevante	7
7.2 Flujo de petición	7
8. Modelo de datos	8
8.1 Relaciones	8
8.2 Reglas de negocio observables	8
9. Diagrama entidad-relación	8
10. Diagrama de clases	10
11. Funcionalidades principales	11
11.1 Parte pública	11
11.2 Administración	11
12. API REST	11
12.1 Ejemplo de body para crear línea de carrito	11
12.2 Códigos de respuesta observados	12
12.3 Validación manual de endpoints	12

13. Vistas web y panel de administración	12
13.1 Rutas de administración	12
13.2 Descripción de las vistas principales	13
14. Autenticación y roles	13
14.1 Roles	13
14.2 Integración del flujo web	13
15. Carrito y checkout simulado	13
15.1 Comportamiento esperado del carrito	13
16. Pruebas y validación manual	13
16.1 Limitación de la validación	14
17. Problemas encontrados y decisiones técnicas	14
17.1 Integración por ramas	14
17.2 Dependencias entre entidades	14
17.3 Uso de Sequelize	14
17.4 Vistas con EJS	14
17.5 Evolución de la documentación	14
18. Limitaciones actuales	14
19. Mejoras futuras	15
20. Conclusión	15
21. Anexos	15
21.1 Puesta en marcha	15
21.2 Variables de entorno	16
21.3 Fuentes consultadas	16

3. Introducción

El Puente Market es una aplicación web de supermercado online desarrollada como proyecto de equipo en un contexto académico de bootcamp. El objetivo no ha sido crear una plataforma de producción, sino construir una base funcional que permita demostrar competencias de backend, integración con base de datos, renderizado de vistas y trabajo colaborativo con GitHub.

La aplicación se apoya en un backend con Node.js y Express, una base de datos PostgreSQL gestionada mediante Sequelize, una API REST para las entidades principales y vistas web renderizadas con EJS. En el estado final indicado, el proyecto incluye catálogo de productos, categorías, usuarios, carrito, checkout simulado, login web con sesiones, autenticación API con JWT y un panel de administración.

La memoria mantiene un tono honesto: se explica lo construido, pero también se indican limitaciones relevantes, como la ausencia de tests automáticos, la necesidad de migraciones versionadas y la conveniencia de reforzar la validación de entradas antes de considerar el proyecto listo para un entorno profesional.

4. Objetivos del proyecto

El objetivo general del proyecto ha sido desarrollar una aplicación web de supermercado online que conecte una interfaz de usuario con un backend estructurado y una base de datos relacional.

4.1 Objetivos funcionales

- Mostrar un catálogo de productos al cliente.
- Organizar los productos por categorías.
- Permitir consultar el detalle de un producto.
- Permitir añadir, modificar y eliminar productos del carrito.
- Simular un proceso de checkout.
- Ofrecer un panel de administración para productos, categorías, usuarios y carrito.
- Persistir la información principal en PostgreSQL.
- Incorporar autenticación web por sesión y autenticación API mediante JWT.

4.2 Objetivos técnicos

- Separar la lógica de la aplicación en rutas, controladores, servicios y modelos.
- Usar Sequelize para definir modelos y relaciones con PostgreSQL.
- Diseñar endpoints REST para los recursos principales.
- Renderizar vistas con EJS y reutilizar layouts.
- Trabajar por ramas funcionales e integrar los cambios mediante PRs.
- Documentar el proceso, las decisiones técnicas y las evidencias de validación manual.

5. Tecnologías utilizadas

La siguiente tabla recoge las tecnologías documentadas en el proyecto y su uso principal dentro de la aplicación.

Tecnología	Uso en el proyecto
Node.js	Entorno de ejecución del backend.
Express	Servidor HTTP, rutas web y API REST.
PostgreSQL	Base de datos relacional para persistir productos, categorías, usuarios y carrito.
Sequelize	ORM para definir modelos, relaciones y sincronización con la base de datos.

Tecnología	Uso en el proyecto
EJS	Motor de plantillas para vistas renderizadas desde servidor.
express-ejs-layouts	Layouts reutilizables para páginas públicas, autenticación y panel de administración.
express-session	Gestión de sesión web para login y acceso a vistas protegidas.
bcrypt	Hash y comprobación de contraseñas.
jsonwebtoken	Generación y validación de JWT para la API.
dotenv	Gestión de variables de entorno.
nodemon	Recarga automática del servidor en desarrollo.
Git + GitHub	Trabajo por ramas, commits, pull requests e integración final.

Nota: el README de la rama principal incluye el stack base, mientras que el package.json de main refleja dependencias adicionales del estado final, como EJS, sesiones, bcrypt y JWT. La memoria toma como referencia el estado final indicado por el usuario y la rama de documentación.

6. Organización del equipo y proceso de trabajo con GitHub

El equipo de trabajo estuvo formado por Jon, Ane, Edy y David, dentro del Bootcamp FullStack The Bridge BBK. El desarrollo se organizó combinando Jira para la gestión de tareas y GitHub para el control de versiones, revisión e integración del código.

El desarrollo se organizó por ramas funcionales. Primero se creó la base del proyecto Express y la conexión con PostgreSQL. Después se repartieron las entidades principales entre diferentes ramas: usuario, categoría, producto y carrito. Más adelante se añadieron servicios, controladores, vistas públicas, panel de administración y flujo de autenticación.

6.1 Uso de Jira para la gestión de tareas

Durante el proyecto se utilizó Jira como herramienta de organización del trabajo. El tablero del proyecto PUENTE sirvió para dividir el alcance en tareas, dar seguimiento al avance y relacionar parte del trabajo técnico con identificadores como PUENTE-21, PUENTE-25, PUENTE-40 o PUENTE-51, visibles también en nombres de ramas y documentación.

El enlace del tablero Jira aportado por el equipo requiere acceso al espacio de Atlassian, por lo que en esta memoria no se detallan asignaciones internas ni estados exactos de tickets que no estén reflejados en el README, las ramas o la documentación del repositorio. Se documenta su uso como herramienta de planificación y seguimiento, sin inventar información no verificable.

Uso de Jira	Aplicación en el proyecto
Planificación	Dividir el proyecto en tareas manejables y relacionarlas con el alcance funcional.
Seguimiento	Visualizar el estado de las tareas durante el desarrollo y la integración.
Trazabilidad	Usar identificadores PUENTE en ramas, PRs y documentación para conectar tarea y cambio técnico.
Coordinación	Facilitar el reparto de trabajo entre Jon, Ane, Edy y David.

6.2 Fases de trabajo

Fase	Área	Descripción
Fase 1	Base backend	Creación del proyecto Express, configuración de Sequelize, conexión a PostgreSQL y primeros endpoints.
Fase 2	Modelos y servicios	Definición de entidades, relaciones y separación de lógica en capa service.
Fase 3	API REST	Rutas REST agrupadas bajo /api para categorías, productos, usuarios, carrito y auth.
Fase 4	Vistas web	Catálogo, detalle de producto, categorías, sidebar de carrito, checkout y panel de administración.
Fase 5	Integración final	Unificación de ramas, resolución de conflictos y PR final hacia main.

6.3 Hitos de integración

Hito	Rama / PR	Descripción
Base inicial	PUENTE-21/creacion-proyecto	Creación del proyecto Express.
Modelos y BBDD	PUENTE-13, PUENTE-14, PUENTE-15, PUENTE-16	Tablas principales y relaciones.
Servicios	PUENTE-41, PUENTE-42, PUENTE-43, PUENTE-44	Separación de lógica en capa service.
Vistas dashboard	PUENTE-40/creacion-vistas-dash	Panel de administración.
Vistas públicas	PUENTE-35, PUENTE-37, PUENTE-38	Productos, carrito y categorías.
Integración vistas	PUENTE-23/vistas	Integración principal de vistas.
Integración compra	task/integracion-productos-categorias	Carrito, compra y flujo público.
Pruebas de integración	PUENTE-51/test-integracion-vistas	Rama usada para validar integración de vistas antes de PR.
Merge final	PR dev -> main	Estado final oficial del proyecto.

6.4 Criterio de integración

La documentación de integración recomendaba evitar merges masivos. El proceso más seguro era integrar las ramas en orden, actualizando cada rama con el último estado de dev antes de abrir o cerrar la PR. Este enfoque reducía conflictos en archivos compartidos como modelos, router API y README.

- Integrar primero categoría, porque producto depende de ella.
- Integrar producto antes de carrito, porque cart depende de ProductModel.
- Integrar usuario antes de carrito, porque cart depende de UserModel.
- Dejar carrito como rama integradora al final por sus dependencias con usuario y producto.

7. Arquitectura de la aplicación

La aplicación sigue una arquitectura por capas sencilla. Esta decisión encaja con un proyecto académico porque permite dividir el trabajo, localizar responsabilidades y evitar que toda la lógica quede concentrada en los archivos de rutas.

Capa / carpeta	Responsabilidad
config	Conexión a base de datos y funciones de comprobación/sincronización.
routes	Define los endpoints y delega en controladores. Incluye rutas API y rutas de vistas.
controllers	Recibe la petición HTTP, valida el flujo básico y prepara la respuesta.

Capa / carpeta	Responsabilidad
services	Encapsula operaciones sobre datos y reduce el acoplamiento con Sequelize.
models	Define entidades y relaciones Sequelize.
middlewares	Autenticación, roles, validaciones e inyección de usuario en vistas.
views	Plantillas EJS para páginas públicas, autenticación y administración.
public	Recursos estáticos: estilos, imágenes y scripts de interfaz.
docs	Documentación de apoyo y evidencias del proyecto.

7.1 Estructura relevante

```
src/
config/ conexión a base de datos
controllers/ controladores API y vistas
middlewares/ autenticación, roles y validaciones
models/ modelos Sequelize
routes/ rutas API y rutas web
services/ lógica de acceso a datos
views/ plantillas EJS
public/ estilos, imágenes y scripts de interfaz
docs/ documentación y evidencias del proyecto
```

7.2 Flujo de petición

El flujo general de una petición separa la entrada HTTP, la lógica de negocio y el acceso a base de datos. En una petición típica, el router dirige la solicitud al controlador, el controlador llama al servicio y el servicio opera con el modelo Sequelize.

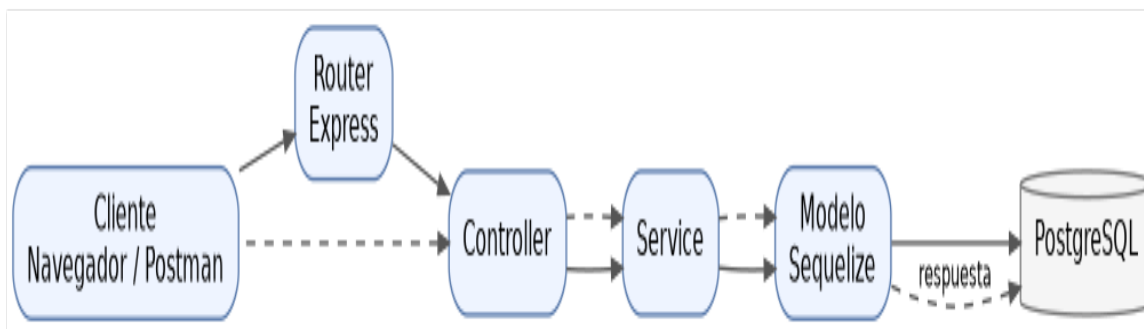


Figura 1. Flujo de petición renderizado a partir del diagrama lógico.

```

```mermaid
flowchart LR
 cliente[Cliente / Navegador / Postman]
 router[Router Express]
 controller[Controller]
 service[Service]
 model[Modelo Sequelize]
 db[(PostgreSQL)]

 cliente --> router
 router --> controller
 controller --> service
 service --> model
 model --> db
 db -- respuesta --> model
 cliente -.-> controller
  ```
```

8. Modelo de datos

El dominio principal del proyecto está formado por cuatro entidades: user, category, product y cart. La entidad cart funciona como línea de carrito y relaciona usuarios con productos, almacenando cantidad y total acumulado por línea.

| Entidad | Responsabilidad | Campos principales |
|----------|--|---|
| category | Categorías del catálogo. | id, name |
| product | Productos de la tienda. | id, id_category, name, price, description, image, stock, on_discount, price_discount, created_at |
| user | Usuarios registrados o gestionados en base de datos. | dni, username, email, password, name, surname, address, phone, card_number, valid_date, cvv, type, created_at |
| cart | Línea de carrito que vincula usuario y producto. | id, user_dni, product_id, quantity, total_amount, updated_at, created_at |

8.1 Relaciones

- Una categoría puede tener muchos productos.
- Un producto pertenece a una categoría.
- Un usuario puede tener muchas líneas de carrito.
- Un producto puede aparecer en muchas líneas de carrito.
- Cart actúa como tabla intermedia entre user y product, con cantidad y total acumulado.

8.2 Reglas de negocio observables

- product.id_category debe apuntar a una categoría existente.
- cart.user_dni debe apuntar a un usuario existente.
- cart.product_id debe apuntar a un producto existente.
- quantity representa la cantidad de producto en una línea del carrito.
- total_amount representa el total acumulado de esa línea.

9. Diagrama entidad-relación

El siguiente diagrama resume las entidades principales y sus relaciones. La versión visual se incluye para lectura rápida y el bloque Mermaid permite mantener el diagrama documentado de forma reproducible.

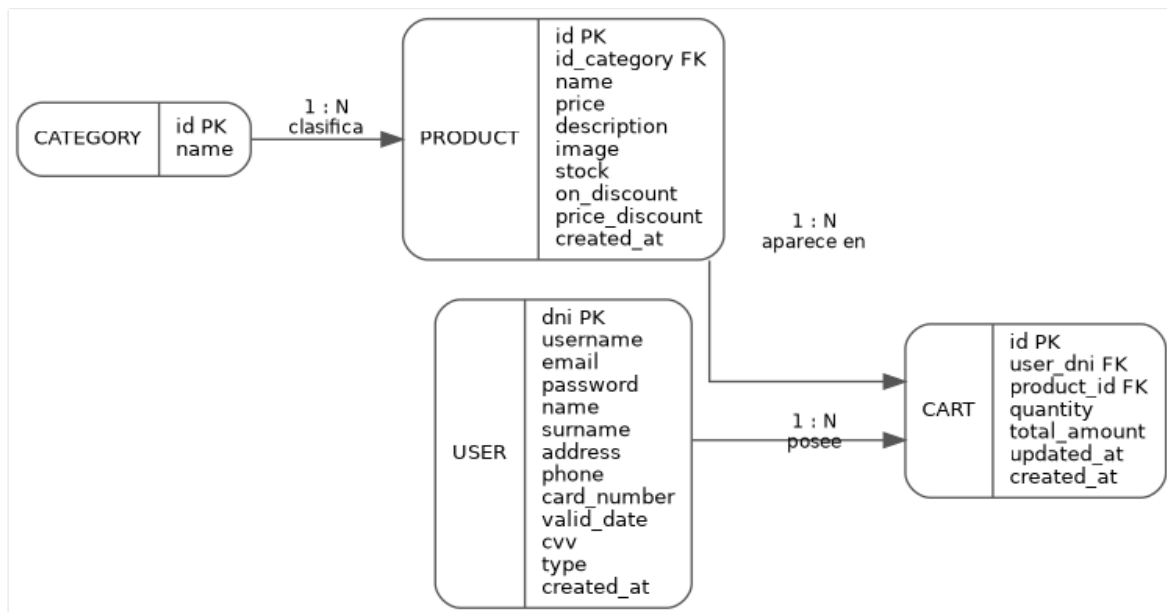


Figura 2. Diagrama entidad-relación simplificado.

```

```mermaid
erDiagram
CATEGORY ||--o{ PRODUCT : clasifica
USER ||--o{ CART : posee
PRODUCT ||--o{ CART : aparece_en

CATEGORY {
INT id PK
STRING name
}
PRODUCT {
INT id PK
INT id_category FK
STRING name
DECIMAL price
STRING description
STRING image
INT stock
BOOLEAN on_discount
DECIMAL price_discount
DATE created_at
}
USER {
STRING dni PK
STRING username
STRING email
STRING password
STRING name
STRING surname
STRING address
STRING phone
BIGINT card_number
DATE valid_date
INT cvv
STRING type
DATE created_at
}
CART {
INT id PK
STRING user_dni FK
INT product_id FK
INT quantity
DECIMAL total_amount
DATE updated_at
DATE created_at
}
```

```

10. Diagrama de clases

El diagrama de clases simplificado muestra los modelos principales y una parte de la relación entre controladores, servicios y modelos. No pretende documentar todos los métodos internos, sino representar la estructura general del proyecto.

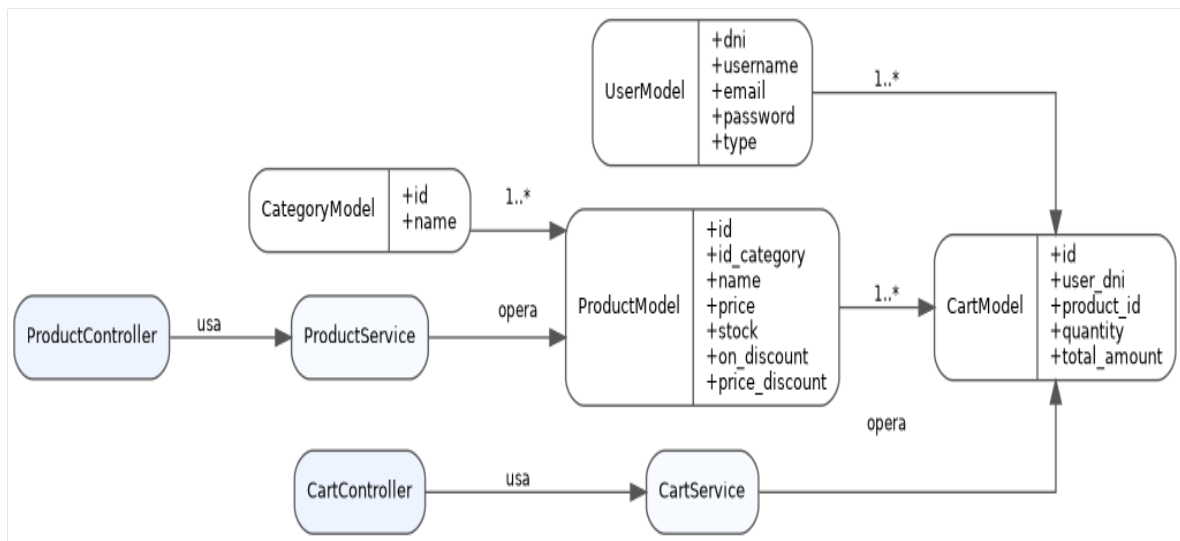


Figura 3. Diagrama de clases simplificado.

```
```mermaid
classDiagram
class UserModel {
+dni
+username
+email
+password
+type
}
class CategoryModel {
+id
+name
}
class ProductModel {
+id
+id_category
+name
+price
+stock
+on_discount
+price_discount
}
class CartModel {
+id
+user_dni
+product_id
+quantity
+total_amount
}
class ProductService
class CartService
class ProductController
class CartController
CategoryModel "1" --> "*" ProductModel
UserModel "1" --> "*" CartModel
ProductModel "1" --> "*" CartModel
ProductController --> ProductService
CartController --> CartService
ProductService --> ProductModel
CartService --> CartModel
```
```

11. Funcionalidades principales

11.1 Parte pública

La parte pública permite navegar por el catálogo de supermercado, consultar productos y operar con el carrito de compra. El flujo se ha planteado como una simulación académica de e-commerce, no como una pasarela de pago real.

- Ver productos disponibles.
- Filtrar productos por categoría.
- Consultar el detalle de un producto.
- Añadir productos al carrito.
- Abrir el sidebar del carrito.
- Modificar cantidades.
- Eliminar productos del carrito.
- Simular el proceso de pago.

11.2 Administración

El panel de administración permite gestionar las entidades principales del proyecto. Está pensado para usuarios con tipo admin y queda protegido mediante sesión y middleware de rol en las rutas correspondientes.

- Gestión de usuarios.
- Gestión de productos.
- Gestión de categorías.
- Gestión de carritos.

12. API REST

La API REST se monta bajo la ruta base local `http://localhost:3000/api`. Permite operar con autenticación, usuarios, productos, categorías y carrito. La validación de entrada no está documentada de forma uniforme en todos los recursos, por lo que una mejora futura sería definir una especificación OpenAPI/Swagger.

| Recurso | Rutas principales |
|------------|---|
| Auth | POST /api/auth/register; POST /api/auth/login |
| Usuarios | GET /api/user; GET /api/user/:dni; POST /api/user; PUT /api/user/:dni; DELETE /api/user/:dni |
| Productos | GET /api/product; GET /api/product/category/:id; POST /api/product; PUT /api/product/:id; PATCH /api/product/:id; DELETE /api/product/:id |
| Categorías | GET /api/category; GET /api/category/:id; POST /api/category; PUT /api/category/:id; DELETE /api/category/:id |
| Carrito | GET /api/cart; GET /api/cart/user/:userDni; GET /api/cart/:id; POST /api/cart; PATCH /api/cart/:id; DELETE /api/cart/:id |

12.1 Ejemplo de body para crear línea de carrito

```
{
  "user_dni": "12345678A",
  "product_id": 2,
  "quantity": 3,
  "total_amount": 2.97
}
```

12.2 Códigos de respuesta observados

| Código | Uso documentado |
|---------------------------|---|
| 200 OK | Lecturas, actualizaciones o borrados correctos. |
| 201 Created | Creaciones correctas. |
| 400 Bad Request | Faltan datos obligatorios o la actualización no incluye campos. |
| 404 Not Found | El recurso solicitado no existe. |
| 500 Internal Server Error | Error no controlado. |

12.3 Validación manual de endpoints

La validación de la API se documenta de forma textual. Para una entrega posterior, podría añadirse una colección de Postman o una especificación OpenAPI, pero esta versión del documento se mantiene únicamente en texto, tablas y diagramas.

13. Vistas web y panel de administración

Además de la API, el proyecto incluye vistas EJS renderizadas en servidor. Esto permite presentar un catálogo público y un panel de administración sin depender de un frontend SPA separado.

| Método | Ruta | Descripción |
|--------|-----------------|--|
| GET | / | Catálogo principal. |
| GET | /:id | Detalle público de producto. Ruta funcional pero mejorable por ambigüedad. |
| GET | /category/:id | Productos de una categoría. |
| GET | /auth/login | Formulario de login. |
| POST | /auth/login | Procesar login web. |
| GET | /auth/register | Formulario de registro. |
| POST | /auth/register | Procesar registro. |
| POST | /auth/logout | Cerrar sesión. |
| POST | /carrito | Añadir producto al carrito. |
| POST | /carrito/addOne | Incrementar o reducir cantidad. |
| POST | /carrito/delete | Eliminar item del carrito. |
| GET | /checkout | Resumen de pago. |
| POST | /checkout | Simulación de pago. |

13.1 Rutas de administración

| Método | Ruta | Descripción |
|--------|-----------------|------------------------|
| GET | /admin | Dashboard principal. |
| GET | /admin/user | Gestión de usuarios. |
| GET | /admin/product | Gestión de productos. |
| GET | /admin/category | Gestión de categorías. |
| GET | /admin/cart | Gestión de carrito. |

13.2 Descripción de las vistas principales

La documentación de vistas se mantiene en formato textual para que el PDF sea más ligero y no dependa de imágenes externas. Las pantallas principales descritas son el catálogo público, el detalle de producto, el login, el carrito y el dashboard de administración.

14. Autenticación y roles

El proyecto contiene dos flujos de autenticación: uno web, basado en express-session, y otro para API, basado en jsonwebtoken. Las contraseñas se almacenan hasheadas mediante bcrypt.

| Flujo | Tecnología | Uso |
|------------|-----------------|---|
| Web | express-session | Acceso a vistas protegidas y dashboard. |
| API | jsonwebtoken | Login API y autorización mediante Bearer token. |
| Contraseña | bcrypt | Hash y comprobación de credenciales. |

14.1 Roles

El panel de administración está pensado para usuarios con tipo admin. El middleware de rol comprueba el tipo guardado en la sesión y redirige a login cuando el usuario no está autenticado o no cumple el rol esperado.

14.2 Integración del flujo web

La evidencia de la rama PUENTE-51 indica que el login web fue ajustado para redirigir a /admin cuando el usuario es admin y a / cuando el usuario es cliente. También se validó que /admin sin sesión redirige a login y que el logout queda operativo.

15. Carrito y checkout simulado

El carrito es una de las funcionalidades centrales del proyecto. Se modela mediante la entidad cart, que relaciona un usuario con un producto y almacena la cantidad y el total de la línea.

- GET /api/cart permite consultar líneas de carrito.
- GET /api/cart/user/:userDni permite consultar el carrito de un usuario.
- POST /api/cart permite crear una línea de carrito.
- PATCH /api/cart/:id permite modificar parcialmente una línea.
- DELETE /api/cart/:id permite eliminar una línea existente.
- Las vistas permiten añadir, incrementar/reducir y eliminar productos del carrito.
- El checkout es una simulación: no existe pasarela de pago real ni cobro efectivo.

15.1 Comportamiento esperado del carrito

El carrito agrupa las líneas asociadas a un usuario, permite modificar cantidades y recalculan el total de forma simulada. La versión actual no incluye integración con pasarela de pago real.

16. Pruebas y validación manual

La validación del proyecto se ha realizado principalmente de forma manual durante la integración. Este enfoque es habitual en fases iniciales de un proyecto académico, pero no sustituye a una batería de tests automatizados.

| Área | Validación realizada |
|---------------|---|
| Base de datos | Conexión con PostgreSQL mediante Sequelize. |
| Modelos | Sincronización del esquema al arrancar la aplicación. |

| Área | Validación realizada |
|-------------------|--|
| Categorías | GET /api/category responde correctamente. |
| Carrito | GET /api/cart, GET /api/cart/user/:userDni, POST /api/cart y DELETE /api/cart/:id validados manualmente. |
| Vistas | Renderizado de EJS, navegación por catálogo y categorías. |
| Autenticación web | Login, logout, redirección por sesión y acceso admin validado en integración. |
| Integración | PR final hacia main y pruebas de ramas de integración. |

16.1 Limitación de la validación

El proyecto no incluye todavía scripts automáticos de test ni lint. Por tanto, la validación depende de comprobaciones manuales y de datos reales o de prueba en la base de datos local. Esta limitación se recoge como mejora futura prioritaria.

17. Problemas encontrados y decisiones técnicas

17.1 Integración por ramas

El trabajo por ramas permitió repartir tareas, pero también generó conflictos en zonas compartidas del código, especialmente modelos, rutas y documentación. La solución propuesta fue integrar de forma secuencial y actualizar cada rama con dev antes de cerrar su PR.

17.2 Dependencias entre entidades

El orden de integración no era indiferente: producto dependía de categoría y carrito dependía tanto de usuario como de producto. Por eso la rama de carrito se trató como una rama más integradora y se recomendó incorporarla al final de la primera fase de modelos.

17.3 Uso de Sequelize

Se eligió Sequelize porque permite definir modelos y asociaciones desde JavaScript. Esto acelera el desarrollo inicial y encaja con el nivel del proyecto. La principal contrapartida es que el uso de `sync({ alter: true })` no sustituye una estrategia formal de migraciones.

17.4 Vistas con EJS

EJS se utilizó para cumplir el objetivo de vistas renderizadas desde servidor y para reutilizar layouts y partials. Esta decisión reduce la complejidad frente a un frontend separado, aunque limita la experiencia respecto a una SPA moderna.

17.5 Evolución de la documentación

Algunos documentos de apoyo reflejan fases anteriores del proyecto, por ejemplo cuando todavía se indicaba que no existía autenticación completa. El estado final indicado y los archivos de main muestran que se añadieron sesiones, roles y JWT. Esta memoria toma el estado final como referencia y conserva las limitaciones antiguas solo cuando siguen siendo aplicables.

18. Limitaciones actuales

- No existen scripts automáticos de test ni lint en el estado documentado.
- La API no dispone todavía de especificación OpenAPI/Swagger.
- La validación de entrada no está documentada de forma uniforme en todos los endpoints.
- El uso de `sequelize.sync({ alter: true })` es útil en desarrollo, pero no es una estrategia robusta de migraciones.

- La ruta pública `/:id` para detalle de producto puede resultar ambigua y debería sustituirse por una ruta más explícita como `/product/:id`.
- El checkout es una simulación y no integra pagos reales.
- Conviene revisar el tratamiento de campos sensibles del usuario, especialmente datos de tarjeta.

19. Mejoras futuras

Las mejoras futuras se plantean como una evolución razonable del proyecto, no como requisitos ya cumplidos. La prioridad debería ser estabilizar lo existente antes de ampliar demasiado el alcance.

| Prioridad | Mejora | Descripción |
|-----------|-------------------|---|
| Alta | Tests automáticos | Añadir pruebas para servicios, controladores y rutas principales. |
| Alta | Migraciones | Sustituir <code>sync({ alter: true })</code> por migraciones versionadas. |
| Alta | Validación | Unificar validaciones de entrada y mensajes de error. |
| Media | OpenAPI | Documentar la API con Swagger/OpenAPI. |
| Media | Rutas | Cambiar rutas ambiguas como <code>/:id</code> por rutas explícitas. |
| Media | Autenticación | Unificar mejor flujo web y API, reforzando roles y permisos. |
| Media | Seguridad | Revisar datos sensibles de usuario y evitar almacenar información de tarjeta innecesaria. |
| Baja | UX | Mejorar diseño visual, mensajes de feedback y estados vacíos. |
| Baja | Datos semilla | Crear seed reproducible para demostraciones locales. |

20. Conclusión

El Puente Market ha evolucionado desde una base inicial de Express y PostgreSQL hasta una aplicación web académica con API REST, vistas EJS, panel de administración, autenticación y carrito de compra. El trabajo principal del equipo ha consistido en conectar piezas desarrolladas por ramas: modelos, servicios, controladores, rutas, vistas y estilos.

La versión final en main representa una base funcional de e-commerce de supermercado. No debe presentarse como una solución de producción, pero sí como un proyecto demostrable que cubre los objetivos principales de backend, base de datos, trabajo colaborativo y construcción de vistas servidor.

La principal conclusión técnica es que el proyecto cumple el objetivo académico de integración, aunque todavía necesita mejoras de calidad: tests, migraciones, documentación formal de API, validación homogénea y revisión de seguridad de datos sensibles.

21. Anexos

21.1 Puesta en marcha

Instalación de dependencias:

```
npm install
```

Arranque en modo desarrollo:

```
npm run dev
```

Arranque normal:

```
npm start
```

URL local por defecto:

```
http://localhost:3000
```

21.2 Variables de entorno

```
PORT=3000
DB_NAME=el_puente_db
DB_USER=admin
DB_PASSWORD=
DB_HOST=localhost
DB_PORT=5432
SESSION_SECRET=
JWT_SECRET=
```

21.3 Fuentes consultadas

| Fuente | Referencia |
|---------------------------|---|
| README main | https://github.com/David-LS-Bilbao/el-puente-market/tree/main |
| README documentación | https://github.com/David-LS-Bilbao/el-puente-market/tree/PUENTE-25/documentacion |
| Documentos docs/ | 00-resumen-proyecto, 01-requisitos-y-casos-de-uso, 02-arquitectura-aplicacion, 03-modelo-datos, 04-api, 05-pruebas-y-validacion, 06-decisiones-tecnicas, evidencias.md, integracion_ramas.md |
| Jira | Proyecto PUENTE en Atlassian. Enlace revisado, pero sin acceso público al detalle interno del tablero. |
| Archivos fuente revisados | package.json, src/index.js, src/routes/router.js, src/routes/api/apiRouter.js, src/routes/views/viewRouter.js, src/routes/views/cartViewRouter.js, src/models/index.js, src/middlewares/authMiddleware.js |

Los datos no documentados de forma explícita no se han completado de manera inventada. La memoria incorpora los nombres facilitados por el equipo y el uso de Jira como herramienta de gestión, pero no inventa estados, asignaciones ni comentarios internos del tablero si no aparecen en fuentes accesibles.